

P1030R3: `std::filesystem::path_view`

Document #: P1030R3
Date: 2019-09-26
Project: Programming Language C++
Library Evolution Working Group
Reply-to: Niall Douglas
<s_sourceforge@nedprod.com>

A proposal for a `std::filesystem::path_view`, a non-owning view of explicitly unencoded or encoded character sequences in the format of a local filesystem path, or a view of a binary key.

A mostly-conforming reference implementation of the proposed path view can be found at https://github.com/ned14/llfio/blob/master/include/llfio/v2.0/path_view.hpp. It has been found to work well on recent editions of GCC, clang and Microsoft Visual Studio, on x86, x64, ARM and AArch64. It has been in production use for several years now.

Changes since R2 due to LEWG and SG16 Unicode feedback:

- A new `path_view_component` prevents Ranges getting confused when iterating a path view.
- `char` source has been restored, it is the narrow system encoding.
- `byte` input has had its specification strengthened.
- Peeking off the end of input has been removed, now construction supplies whether input is zero terminated or not.
- `c_str` can now generate many renditions of the path view via template parameter, and the relationship to the filesystem native encoding has been weakened.
- Relative comparison operator overloads have been removed, as comparison is very expensive, and anyone using path views in say a `std::map` should always define a custom comparator (which is more efficient).
- Equality comparisons are now identity-based instead of lexicographic.
- After many, many exchanges with SG16 about how best to tame the evil of comparing filesystem paths, I have come up with a whole new way of doing path view comparison which hopefully ticks everybody's boxes.
- Some asked for visitation of the source data, added.
- Default stack internal buffer size has been reduced to 1Kb characters.

Contents

1	Introduction	2
2	Impact on the Standard	3
3	Proposed Design	3

3.1	<code>path_view_component</code>	4
3.2	<code>path_view</code>	7
3.3	Example of use	11
4	Design decisions, guidelines and rationale	11
4.1	Fixed use of stack in <code>struct c_str</code>	12
4.2	Path view consumer determines the path interpretation semantics	13
5	Technical specifications	15
6	Frequently asked questions	15
6.1	Does this mean that all APIs consuming <code>std::filesystem::path</code> ought to now consume <code>std::filesystem::path_view</code> instead?	15
7	Acknowledgements	15
8	References	15

1 Introduction

In the current C++ standard, the canonical way for supplying filesystem paths to C++ functions which consume file system paths is `std::filesystem::path`. This wraps up `std::filesystem::path::string_type` (= `std::basic_string<Char>`) with a platform specific choice of `Char` (currently Microsoft Windows uses `Char = wchar_t`, everything else uses `Char = char`) with iterators and member functions which parse the string according to the path delimiters for that platform. For example `std::filesystem::path` on Microsoft Windows might parse this string:

C:\Windows\System32\notepad.exe

into:

- `root_name()` = "C:"
- `root_directory()` = "\"
- `root_path()` = "C:\"
- `relative_path()` = "Windows\System32\notepad.exe"
- `parent_path()` = "C:\Windows\System32"
- `filename()` = "notepad.exe"
- `stem()` = "notepad"
- `extension()` = ".exe"
- `*begin()` = "C:"

- `*++begin()` = `“/”` (note the forward, not backward, slash. This is considered to be the name of the root directory)
- `*++++begin()` = `“Windows”`
- `*+++++begin()` = `“System32”` (note no intervening slash)

For every one of these decompositions, a new `path` is returned, which means a new underlying `std::basic_string<Char>`, which means a new memory allocation. In code which performs a lot of path traversal and decomposition, these memory allocations, and the copying of fragments of path around, can start to add up. For example, in [P1031] *Low level file i/o library*, a directory enumeration costs around 250 *nanoseconds* per entry amortised. Each path construction might cost that again. Therefore, for each item enumerated, one *halves* the directory enumeration performance solely due to the choice of `path`, which is why P1031 uses `path_view` instead, and thus can enumerate four million directory items per second, which makes handling ten million item plus directories tractable.

There is also a negative effect on CPU caches of copying around path strings. Paths are increasingly reaching hundred of bytes, as anyone running into the 260 path character limit on Microsoft Windows can testify¹. Every time one copies a path, one is evicting potentially useful data from the CPU caches, which need not be evicted if one did not copy paths.

Enter thus the proposed `std::filesystem::path_view`, which is a lightweight reference to part, or all of, a source of filesystem path data. It provides most of the same member functions as `std::filesystem::path`, operating by constant and often constexpr reference upon some character source which is in the format of the local platform’s file system path, or a generic path, same as with `std::filesystem::path`. It is intended that for most functions currently accepting a `std::filesystem::path`, they can now accept a `std::filesystem::path_view` instead with minor to none refactoring of implementation.

2 Impact on the Standard

The proposed library is a pure-library solution.

3 Proposed Design

Much of the proposed path view is unsurprising, with a large subset of `std::filesystem::path`’s observers and modifiers replicated (apart from `path`’s mutating functions, which here are non-mutating and return new views instead). Constexpr abounds, and the path view is trivially copyable and is thus suitable for passing around by value.

WG21 feedback suggested that iteration of path views ought to not return another path view, so iteration returns `path_view_component` instead. I appreciate that this is a large divergence from filesystem path, however feedback suggests that filesystem path is deficient in this regard.

¹You can now build your Windows application with this limit removed for your program.

Path views represent a user unknown polymorphic view of characters or bytes. The proposed supported path source encodings are:

1. `char`, the narrow native system encoding.
2. `wchar_t`, the wide native system encoding.
3. `char8_t`, UTF-8 encoding.
4. `char16_t`, UTF-16 encoding.
5. `byte`, raw encoded or unencoded bytes. This can mean ‘passthrough’ for some consumers of path views, or may take on some other meaning depending on consumer.

3.1 `path_view_component`

Path view components look very much like path views, but do not offer path component iteration, nor any of the path interpretation member functions based upon the filesystem path separator.

```
1 class path_view_component
2 {
3 public:
4     /// True if path views can be constructed from this character type.
5     /// i.e. is one of 'char', 'wchar_t', 'char8_t', 'char16_t'
6     template <class Char> static constexpr bool is_source_chartype_acceptable;
7
8     /// True if path views can be constructed from this source.
9     /// i.e. 'is_source_chartype_acceptable', or is 'byte'
10    template <class Char> static constexpr bool is_source_acceptable;
11
12    /// The default internal buffer size used by 'c_str'.
13    static constexpr size_t default_internal_buffer_size = 1024; // 2Kb for wchar_t, 1Kb for char
14
15 public:
16    path_view_component() = default;
17    path_view_component(const path_view_component &) = default;
18    path_view_component(path_view_component &&) = default;
19    path_view_component &operator=(const path_view_component &) = default;
20    path_view_component &operator=(path_view_component &&) = default;
21    ~path_view_component() = default;
22
23    /// True if empty
24    [[nodiscard]] constexpr bool empty() const noexcept;
25    constexpr bool has_stem() const noexcept;
26    constexpr bool has_extension() const noexcept;
27
28    /// Returns the size of the view in characters.
29    constexpr size_t native_size() const noexcept;
30
31    /// Swap the view with another
32    constexpr void swap(path_view_component &o) noexcept;
33
34    // True if the view contains any of the characters '*', '?', (POSIX only: '[' or ']').
35    constexpr bool contains_glob() const noexcept;
36
```

```

37  ///! Returns a view of the filename without any file extension
38  constexpr path_view_component stem() const noexcept;
39
40  ///! Returns a view of the file extension part of this view
41  constexpr path_view_component extension() const noexcept;
42
43  ///! Return the path view as a path. Allocates and copies memory!
44  filesystem::path path() const;
45
46  /*! Compares the two path views for equivalence or ordering using 'T'
47  as the destination encoding, if necessary.
48
49  If the source encodings of the two path views are compatible, a
50  lexicographical comparison is performed. If they are incompatible,
51  either or both views are converted to the destination encoding
52  using 'c_str<T, Deleter, _internal_buffer_size>', and then a
53  lexicographical comparison is performed.
54
55  This can, for obvious reasons, be expensive. It can also throw
56  exceptions, as 'c_str' does.
57
58  If the destination encoding is 'byte', 'memcmp()' is used,
59  and 'c_str' is never invoked as the two sources are byte
60  compared directly.
61  */
62  template <class T = typename filesystem::path::value_type
63           class Deleter = std::default_delete<T[]>,
64           size_t _internal_buffer_size = default_internal_buffer_size
65  >
66  requires(path_view_component::is_source_acceptable<T>)
67  constexpr int compare(const path_view_component &p) const;
68
69  ///! \overload
70  template <class T = typename filesystem::path::value_type
71           class Deleter = std::default_delete<T[]>,
72           size_t _internal_buffer_size = default_internal_buffer_size,
73           class Char
74  >
75  requires(path_view_component::is_source_acceptable<T> && path_view_component::
76           is_source_chartype_acceptable<Char>)
77  constexpr int compare(const Char *s) const;
78
79  ///! \overload
80  template <class T = typename filesystem::path::value_type
81           class Deleter = std::default_delete<T[]>,
82           size_t _internal_buffer_size = default_internal_buffer_size,
83           class Char
84  >
85  requires(path_view_component::is_source_acceptable<T> && path_view_component::
86           is_source_chartype_acceptable<Char>)
87  constexpr int compare(const basic_string_view<Char> s) const;
88
89  /*! Instantiate from a 'path_view_component' to get a path suitable for feeding to other code.
90
91  \tparam T The destination encoding required.
92  \tparam Deleter A custom deleter for any temporary buffer.

```

\tparam _internal_buffer_size Override the size of the internal temporary buffer, thus reducing stack space consumption (most compilers optimise away the internal temporary buffer if it can be proved it will never be used). The default is 1024 values of 'T'.

This makes the input to the path view component into a destination format suitable for consumption by other code. If the source has the same format as the destination, and the zero termination requirements are the same, the source is used directly without memory copying nor reencoding.

If the format is compatible, but the destination requires zero termination, and the source is not zero terminated, a straight memory copy is performed into the temporary buffer.

'c_str' contains a temporary buffer sized according to the template parameter. Output below that amount involves no dynamic memory allocation. Output above that amount calls 'operator new[]'. You can use an externally supplied larger temporary buffer to avoid dynamic memory allocation in all situations.

```
*/
template <class T = typename filesystem::path::value_type,
         class Deleter = std::default_delete<T[]>,
         size_t _internal_buffer_size = default_internal_buffer_size
>
struct c_str
{
    static_assert(is_source_acceptable<T>, "path_view_component::c_str<T> does not have a T which is
        one of byte, char, wchar_t, char8_t nor char16_t");

    //! Type of the value type
    using value_type = T;
    //! Type of the deleter
    using deleter_type = Deleter;
    //! The size of the internal temporary buffer
    static constexpr size_t internal_buffer_size = (_internal_buffer_size == 0) ? 1 :
        _internal_buffer_size;

    //! Number of values, excluding zero terminating char, at buffer
    size_t length{0};
    //! Pointer to the possibly-converted path
    const value_type *buffer{nullptr};

public:
    /*! Construct, performing any reencoding or memory copying required.

    \param view The path component view to use as source.
    \param no_zero_terminate Set to true if zero termination is not required.
    \param allocate A callable with prototype 'value_type *(size_t length)' which
    is defaulted to 'return new value_type[length];'. You can return 'nullptr' if
    you wish, the consumer of 'c_str' will see a 'buffer' set to 'nullptr'.

    If an error occurs during any conversion from UTF-8 or UTF-16, an exception of
    'system_error(errc::illegal_byte_sequence)' is thrown.

    This is because if you tell 'path_view' that its source is UTF-8 or UTF-16, then that
    must be **valid** UTF. If you wish to supply UTF-invalid paths (which are legal
    on most filesystems), use native narrow or wide encoded source, or binary.
    */
```

```

145     template <class U>
146     c_str(const path_view_component &view,
147           bool no_zero_terminate,
148           U &&allocate);
149
150     ///! \overload
151     c_str(const path_view_component &view,
152           bool no_zero_terminate = false);
153
154     ~c_str() = default;
155     c_str(const c_str &) = delete;
156     c_str(c_str &&) = delete;
157     c_str &operator=(const c_str &) = delete;
158     c_str &operator=(c_str &&) = delete;
159
160 private: // For exposition only ...
161     bool _call_deleter{false};
162     Deleter _deleter;
163
164     // MAKE SURE this is the final item in storage, the compiler will elide the storage
165     // under optimisation if it can prove it is never used.
166     value_type _buffer[internal_buffer_size]{};
167 };
168 };
169
170 // These are IDENTITY equality comparisons i.e. equality is same source encoding, same content
171 inline constexpr bool operator==(path_view_component x, path_view_component y) noexcept;
172 inline constexpr bool operator!=(path_view_component x, path_view_component y) noexcept;
173
174 inline std::ostream &operator<<(std::ostream &s, const path_view_component &v);
175
176 // relative comparison disabled
177 // hashing disabled
178
179 // Visitation of source representation, calls f(const T *, size_t, bool)
180 template<class F>
181 inline constexpr auto visit(F &&f, path_view_component);

```

3.2 path_view

```

1 class path_view
2 {
3 public:
4     ///! Const iterator type
5     using const_iterator = path_view_iterator;
6     ///! iterator type
7     using iterator = const_iterator;
8     ///! Reverse iterator
9     using reverse_iterator = std::reverse_iterator<iterator>;
10    ///! Const reverse iterator
11    using const_reverse_iterator = std::reverse_iterator<const_iterator>;
12    ///! Size type
13    using size_type = std::size_t;
14    ///! Difference type

```

```

15     using difference_type = std::ptrdiff_t;
16
17     ///! The preferred separator type
18     static constexpr auto preferred_separator = filesystem::path::preferred_separator;
19
20 public:
21     path_view() = default;
22     path_view(const path_view &) = default;
23     path_view(path_view &&) = default;
24     path_view &operator=(const path_view &) = default;
25     path_view &operator=(path_view &&) = default;
26     ~path_view() = default;
27
28     ///! Implicitly constructs a path view from a path. The input path MUST continue to
29     ///! exist for this view to be valid.
30     path_view(const filesystem::path &v) noexcept;
31
32     ///! Implicitly constructs a path view from a path view component. The input path
33     ///! MUST continue to exist for this view to be valid.
34     path_view(path_view_component v) noexcept;
35
36     ///! Implicitly constructs a path view from a zero terminated 'const char *'.
37     ///! The input string MUST continue to exist for this view to be valid.
38     constexpr path_view(const char *v) noexcept;
39
40     ///! Implicitly constructs a path view from a zero terminated 'const wchar_t *'.
41     ///! The input string MUST continue to exist for this view to be valid.
42     constexpr path_view(const wchar_t *v) noexcept;
43
44     ///! Implicitly constructs a path view from a zero terminated 'const char8_t *'.
45     ///! The input string MUST continue to exist for this view to be valid.
46     constexpr path_view(const char8_t *v) noexcept;
47
48     ///! Implicitly constructs a path view from a zero terminated 'const char16_t *'.
49     ///! The input string MUST continue to exist for this view to be valid.
50     constexpr path_view(const char16_t *v) noexcept;
51
52
53     /*! Constructs a path view from a lengthed array of one of
54     'byte', 'char', 'wchar_t', 'char8_t' or 'char16_t'. The input
55     string MUST continue to exist for this view to be valid.
56     */
57     template<class Char>
58     requires(path_view_component::is_source_acceptable<Char>)
59     constexpr path_view(const Char *v,
60                         size_t len,
61                         bool is_zero_terminated) noexcept;
62
63     /*! Constructs from a basic string if the character type is one of
64     'char', 'wchar_t', 'char8_t' or 'char16_t'.
65     */
66     template<class Char>
67     requires(path_view_component::is_source_chartype_acceptable<Char>)
68     constexpr path_view(const std::basic_string<Char> &v) noexcept;
69
70     /*! Constructs from a basic string view if the character type is one of

```

```

71 'char', 'wchar_t', 'char8_t' or 'char16_t'.
72 */
73 template<class Char>
74 requires(path_view_component::is_source_chartype_acceptable<Char>)
75 constexpr path_view(basic_string_view<Char> v,
76                     bool is_zero_terminated) noexcept;
77
78
79 //! Swap the view with another
80 constexpr void swap(path_view &o) noexcept;
81
82 //! True if empty
83 [[nodiscard]] constexpr bool empty() const noexcept;
84 constexpr bool has_root_path() const noexcept;
85 constexpr bool has_root_name() const noexcept;
86 constexpr bool has_root_directory() const noexcept;
87 constexpr bool has_relative_path() const noexcept;
88 constexpr bool has_parent_path() const noexcept;
89 constexpr bool has_filename() const noexcept;
90 constexpr bool has_stem() const noexcept;
91 constexpr bool has_extension() const noexcept;
92 constexpr bool is_absolute() const noexcept;
93 constexpr bool is_relative() const noexcept;
94
95 // True if the path view contains any of the characters '*', '?', (POSIX only: '[' or ']').
96 constexpr bool contains_glob() const noexcept;
97
98 #ifndef _WIN32
99 // True if the path view is a NT kernel path starting with '\\!\\' or '\\??\\'
100 constexpr bool is_ntpath() const noexcept;
101 #endif
102
103 //! Returns an iterator to the first path component
104 constexpr inline const_iterator cbegin() const noexcept;
105 //! Returns an iterator to the first path component
106 constexpr inline const_iterator begin() const noexcept;
107 //! Returns an iterator to the first path component
108 constexpr inline iterator begin() noexcept;
109 //! Returns an iterator to after the last path component
110 constexpr inline const_iterator cend() const noexcept;
111 //! Returns an iterator to after the last path component
112 constexpr inline const_iterator end() const noexcept;
113 //! Returns an iterator to after the last path component
114 constexpr inline iterator end() noexcept;
115
116 //! Returns a copy of this view with the end adjusted to match the final separator.
117 constexpr path_view remove_filename() const noexcept;
118
119 //! Returns the size of the view in characters.
120 constexpr size_t native_size() const noexcept;
121
122 //! Returns a view of the root name part of this view e.g. C:
123 constexpr path_view root_name() const noexcept;
124
125 //! Returns a view of the root directory, if there is one e.g. /
126 constexpr path_view root_directory() const noexcept;

```

```

127
128 ///! Returns, if any, a view of the root path part of this view e.g. C:/
129 constexpr path_view root_path() const noexcept;
130
131 ///! Returns a view of everything after the root path
132 constexpr path_view relative_path() const noexcept;
133
134 ///! Returns a view of the everything apart from the filename part of this view
135 constexpr path_view parent_path() const noexcept;
136
137 ///! Returns a view of the filename part of this view.
138 constexpr path_view_component filename() const noexcept;
139
140 ///! Returns a view of the filename without any file extension
141 constexpr path_view_component stem() const noexcept;
142
143 ///! Returns a view of the file extension part of this view
144 constexpr path_view_component extension() const noexcept;
145
146 ///! Return the path view as a path. Allocates and copies memory!
147 filesystem::path path() const;
148
149 /*! Compares the two path views for equivalence or ordering using 'T'
150 as the destination encoding, if necessary.
151
152 If the source encodings of the two path views are compatible, a
153 lexicographical comparison is performed. If they are incompatible,
154 either or both views are converted to the destination encoding
155 using 'c_str<T, Delete, _internal_buffer_size>', and then a
156 lexicographical comparison is performed.
157
158 This can, for obvious reasons, be expensive. It can also throw
159 exceptions, as 'c_str' does.
160
161 If the destination encoding is 'byte', 'memcmp()' is used,
162 and 'c_str' is never invoked as the two sources are byte
163 compared directly.
164 */
165 template <class T = typename filesystem::path::value_type
166         class Deleter = std::default_delete<T[]>,
167         size_t _internal_buffer_size = path_view_component::default_internal_buffer_size
168 >
169 requires(path_view_component::is_source_acceptable<T>)
170 constexpr int compare(const path_view_component &p) const;
171
172 ///! \overload
173 template <class T = typename filesystem::path::value_type
174         class Deleter = std::default_delete<T[]>,
175         size_t _internal_buffer_size = path_view_component::default_internal_buffer_size,
176         class Char
177 >
178 requires(path_view_component::is_source_acceptable<T> && path_view_component::
179         is_source_chartype_acceptable<Char>)
180 constexpr int compare(const Char *s) const;
181
182 ///! \overload

```

```

182     template <class T = typename filesystem::path::value_type
183             class Deleter = std::default_delete<T[]>,
184             size_t _internal_buffer_size = path_view_component::default_internal_buffer_size,
185             class Char
186         >
187     requires(path_view_component::is_source_acceptable<T> && path_view_component::
188             is_source_chartype_acceptable<Char>)
189     constexpr int compare(const basic_string_view<Char> s) const;
190
191     //! Instantiate from a 'path_view' to get a path suitable for feeding to other code.
192     //! See 'path_view_component::c_str'.
193     template <class T = typename filesystem::path::value_type,
194             class Deleter = std::default_delete<T[]>,
195             size_t _internal_buffer_size = path_view_component::default_internal_buffer_size
196         >
197     struct c_str
198 };
199
200 // These are IDENTITY equality comparisons i.e. equality is same source encoding, same content
201 inline constexpr bool operator==(path_view x, path_view y) noexcept;
202 inline constexpr bool operator!=(path_view x, path_view y) noexcept;
203
204 inline std::ostream &operator<<(std::ostream &s, const path_view &v);
205
206 // relative comparison disabled
207 // hashing disabled
208
209 // Visitation of source representation, calls f(const T *, size_t, bool)
210 template<class F>
211 inline constexpr auto visit(F &&f, path_view);

```

3.3 Example of use

The use idiom would be as follows:

```

1  int open_file(path_view path)
2  {
3      // I am on POSIX which requires zero terminated char filesystem paths.
4      // So here if the view is zero terminated, and the view refers to
5      // char*, char8_t* or byte data, we can use it directly without memory copying.
6      path_view::c_str<> p(path);
7      return ::open(p.buffer, O_RDONLY);
8  }

```

4 Design decisions, guidelines and rationale

There are a number of non-obvious design decisions in the proposed path view object. These decisions were taken after a great deal of empirical trial and error with ‘more obvious’ designs, where those designs were found wanting in various ways. The author believes that the current set of tradeoffs is close to the ideal set.

The design imperatives for an allocating `std::filesystem::path` are not those for a non-allocating `std::filesystem::path_view`. A ‘handy feature’ of an allocating path object is that it must always copy its input into its allocation. If it is allocating memory and copying the path content in any case, performing an implicit conversion of a native narrow input encoding to say a native wide encoding seems like a reasonable design choice, given the relative cost of the other overheads.

In the case of a path view however, we are trying very hard to not copy memory. If the local platform uses the same narrow or wide input encoding as the source backing the view, and the path view is already terminated by a null character where that is relevant on the local platform, no copying is required. The original is used unmodified, bytes are passed through as-is. Only if necessary, a copy and/or conversion of the input onto the stack is performed into whatever format the local platform requires.

One might argue that in the case of `std::filesystem::path`, we might reuse the path across multiple calls, and thus the path view approach of just-in-time copying per syscall is wasteful on those platforms. However it is exceedingly rare to open the same file more than once, and anyone caring strongly about performance will simply modify their source to use the same native encoding and null termination as the platform.

The next argument is usually one of the form that paths get commonly reused with just the leafname modified, and therefore path’s approach is more efficient as only the leafname gets converted per iteration. I would counter that this proposed path view object comes from [P1031] *Low level file i/o library* where using absolute paths is bad form: you use a `path_handle` to indicate the base directory and supply a path view for the leafname – this is *far* more efficient than any absolute path based mechanism as it avoids the kernel having to traverse the filesystem hierarchy, typically taking a read lock on each inode in the absolute path.

4.1 Fixed use of stack in `struct c_str`

Firstly, note that the compiler elides completely the fixed stack buffer for zero termination and UTF conversions caused by instantiating `struct c_str` if the compiler can prove that it will never be used. So if you supply native format, zero terminated input, to the path view constructor, the compiler should spot that the temporary stack buffer is never used, and thus eliminate it. This ought to be the case most of the time, especially under link time optimisation.

Secondly, the fixed stack buffer tends to get allocated just before a syscall, and released just after that syscall. Stack cache locality is therefore generally unaffected, and the fixed stack buffer does not remain allocated for long. It is thus a once-off stack allocation.

Microsoft Windows systems can have a maximum path of 64Kb, but most paths are likely to be under 1024 codepoints. Of the major POSIX implementations, `PATH_MAX` is 4096 for Linux, MacOS 1024, FreeBSD 1024. All this suggests that a reasonable default for the internal buffer ought to be 1024 codepoints, which is one of 1Kb, 2Kb or 4Kb of stack consumption depending on what the path view consumer is rendering to.

Thus, on Microsoft Windows and Linux only, if the input path exceeds 1024 codepoints, `malloc()` will be used to create a temporary internal buffer. On MacOS and FreeBSD, the internal buffer is always large enough.

For those Linux implementations running on embedded systems where 1Kb stack allocations would be unwise, we do provide for the ability to choose a smaller fixed size buffer in the template parameter, and override the custom allocator to issue a trap if the smaller path limit is exceeded.

Again, I would stress that the programmer can be careful to never send a non-zero terminated string in as a path, and thus completely eliminate the use of temporary buffers on an embedded Linux solution. In any case, path views are considerably less heavy on free RAM than `std::filesystem::path`.

4.2 Path view consumer determines the path interpretation semantics

Path view has been designed around the *consumer* defining what reencoding semantics are in play. For example, a Java JNI might define UTF-16 as the destination encoding for `c_str` irrespective of the native filesystem encoding or platform, and all input is therefore converted to UTF-16 by the JNI's use of `c_str`. This is why `.compare()` is templated exactly as `c_str` is templated, and it is on whomever consumes filesystem path views to define a locally customised `path_view` if the defaults are inappropriate for their use case.

Some have asked for detailed reencoding semantics for the filesystem. Here are those as defined by [P1031] *Low level file i/o*, but let me stress once again that it is the *consumer* of path views which defines how path views are to be interpreted.

These are the path interpretation semantics applied to consuming path views by LLFIO on POSIX:

- `char` = Unix format paths, native filesystem encoding.
- `wchar_t` = Unix format paths (UTF-32). This is converted C++-side to the native filesystem encoding at the point of use, if necessary.
- `char8_t` = Unix format paths (UTF-8). Input must be valid UTF-8. This is converted C++-side to the native filesystem encoding at the point of use, if necessary.
- `char16_t` = Unix format paths (UTF-16). Input must be valid UTF-16. This is converted C++-side to the native filesystem encoding at the point of use, if necessary.
- `byte` = Unique variable width binary number identifier. POSIX does not currently implement a standard API for these kind of paths, but proprietary APIs exist for various filesystems and hardware devices (e.g. ZFS, Samsung KV-SSD).

These are the path interpretation semantics applied to emitting path views by LLFIO on POSIX:

- Directory enumeration produces either the native filesystem encoding in `char`, or a unique variable width binary number identifier in `byte`.

These are the path interpretation semantics applied to consuming path views by LLFIO on Microsoft Windows:

- `char` = Compatibility DOS format paths, narrow system encoding (program locale determined). Compatibility DOS format paths start with `X:\`, or no prefix at all. These call the ANSI editions of Win32 APIs.
- `wchar_t` = Compatibility DOS format paths, wide system encoding (UTF-16). These call the Unicode editions of Win32 APIs.
- `char16_t` = Compatibility DOS format paths in UTF-16. Input must be valid UTF-16. These call the Unicode editions of the Win32 APIs.
- `char8_t` = Compatibility DOS format paths in UTF-8. Input must be valid UTF-8. This is converted C++-side to UTF-16 at the point of use, and the Unicode editions of Win32 APIs are called.
- `char` = Extended DOS format paths, narrow system encoding (program locale determined). As per Win32 API documentation, extended DOS format paths are prefixed with `\\?\` or `\\.\`. These call the ANSI editions of Win32 APIs.
- `wchar_t` = Extended DOS format paths, wide system encoding (UTF-16). These call the Unicode editions of Win32 APIs.
- `char16_t` = Extended DOS format paths in UTF-16. Input must be valid UTF-16. These call the Unicode editions of the Win32 APIs.
- `char8_t` = Extended DOS format paths in UTF-8. Input must be valid UTF-8. This is converted C++-side to UTF-16 at the point of use, and the Unicode editions of Win32 APIs are called.
- `char` = NT format paths, narrow system encoding (program locale determined). This is a LLFIO-only extension, NT format paths are prefixed with `\\!\\`. Paths prefixed with this never use the Win32 APIs, only the NT kernel APIs.
- `wchar_t` = NT format paths, wide system encoding (UTF-16).
- `char16_t` = NT format paths in UTF-16. Input must be valid UTF-16.
- `char8_t` = NT format paths, UTF-8. This is converted C++-side to UTF-16 at the point of use.
- `byte` = Unique variable width binary number identifier (NTFS and ReFS permit a 128-bit key-value lookup of inodes, this may be accelerated in hardware by suitable storage devices).

These are the path interpretation semantics applied to emitting path views by LLFIO on Microsoft Windows:

- Directory enumeration produces either the native filesystem encoding in `wchar_t`, or a unique variable width binary number identifier in `byte`.

5 Technical specifications

No Technical Specifications are involved in this proposal.

6 Frequently asked questions

6.1 Does this mean that all APIs consuming `std::filesystem::path` ought to now consume `std::filesystem::path_view` instead?

Most of the time, perhaps almost always, yes. `std::filesystem::path_view` implicitly constructs from explicitly encoded strings, paths and explicitly encoded string literals. Anywhere you are currently consuming `std::filesystem::path` as a parameter, you can start using `std::filesystem::path_view` instead if this proposal is approved. It would remain the case that where a function is returning a new path, `std::filesystem::path` is the correct choice. So inputs would be mostly path views, outputs would be paths.

Path views can represent more encodings of filesystem paths than `std::filesystem::path` can e.g. unique variable with binary numbers.

This author has replaced paths with path views in an existing piece of complex path decomposition and recomposition, and apart from a few minor source code changes to fix lifetime issues, the code compiled and worked unchanged. Path views are mostly a drop-in replacement for paths, except for when one is creating wholly new paths.

Incidentally, performance of that code improved by approximately twenty fold (20x).

7 Acknowledgements

My thanks to Nicol Bolas, Bengt Gustafsson and Billy O’Neal for their feedback upon this proposal.

8 References

- [P0482] Tom Honermann,
char8_t: A type for UTF-8 characters and strings
<https://wg21.link/P0482>
- [P0882] Yonggang Li
User-defined Literals for std::filesystem::path
<https://wg21.link/P0882>
- [P1031] Douglas, Niall
Low level file i/o library
<https://wg21.link/P1031>